

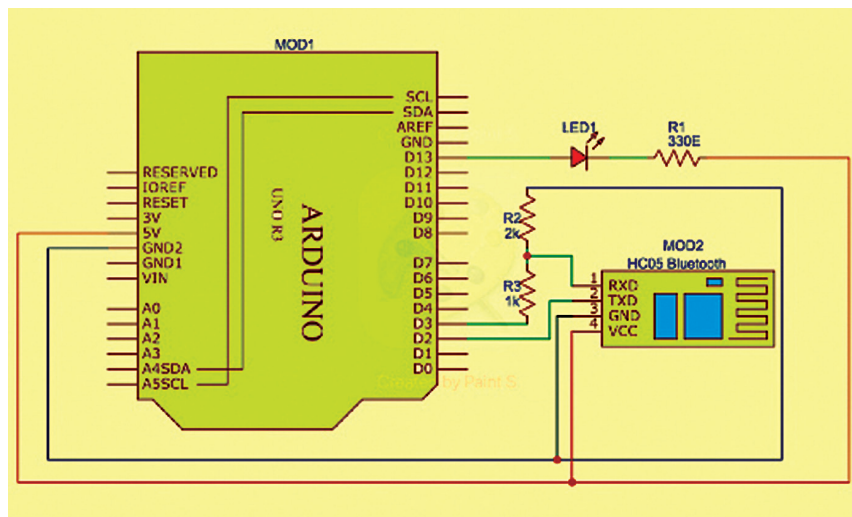


Fig. 3: Circuit diagram of Bluetooth control through mobile

```
#include<SoftwareSerial.h>
#define ledpin 13

SoftwareSerial bt(2,3); /* (Rx,Tx)(of
UNO)*/
//...connect Rx (2) of UNO directly
//...to TXD of BT. Tx (3) of UNO is
//...connected to RXD of BT through
//...voltage divider (1k/2k)
byte state = '2' ;

void setup() {

  delay ( 1000 ) ;
  pinMode ( ledpin , OUTPUT ) ;
  digitalWrite ( ledpin , LOW ) ;

  bt.begin (9600); /* baud rate for SoftwareSerial */
}

void loop() {

  if (bt.available()) { /* If data is available */
    state = bt.read () ;
    if (state == '0')
    { digitalWrite( ledpin , LOW); // Turn LED OFF
      bt.write ( "LED OFF\n" ) ;
      state = '2' ;
    }
    else if ( state == '1' )
    { digitalWrite(ledpin , HIGH);
      bt.write ( "LED ON\n" ) ;
      state = '2' ;
    }
  }
  delay (500) ;
}
```

Fig. 4: Snippet of the source code

is detailed in the Bill of Materials table. An image of the HC-05 Bluetooth module is shown in Fig. 2.

Circuit diagram

Fig. 3 shows the circuit diagram of the Bluetooth control through mobile. It is built around an Arduino

Uno (MOD1), an HC-05 Bluetooth module (MOD2), and a few other components.

In the HC-05, the pin EN stands for enable. This pin, when pulled low, disables the device. When connected to 3.3V or left free, it enables the device. In this setup, it is left free so that it is always enabled. The HC-05 requires a VCC in the range of 3.6V to 6V. Since the Arduino Uno does not provide 3.6V, it is connected to 5V (do not connect to 3.3V as it is too low). The GND of the HC-05 is connected to the GND of the Arduino Uno.

For software serial communication

between the HC-05 and the Uno, the library `SoftwareSerial.h` will be used. Pins 2 and 3 of the Uno will act as RX and TX. Connect RX (pin 2) directly to TXD of the HC-05. Since pin 3 cannot be connected to RXD (as the HC-05 expects 3.3V logic levels), a potential divider

with 1k and 2k resistors is used to connect RXD to pin 3.

The 'State' pin of the HC-05 indicates whether the device is paired with another Bluetooth device. When paired, this pin goes high. In this setup, this pin is not used, so it is left free. The HC-05 also includes an LED that blinks rapidly when not paired and blinks slowly when paired with another device.

Software

The code is written in Arduino IDE. First, the pin for the LED or relay is defined, and then the serial read function is used to receive the command. The command is then used to perform functions like turning the device on or off. Fig. 4 shows a snippet of the source code.

This device requires Bluetooth to send commands to the Arduino. Either Kodular or App Inventor can be used to create the app. In this case, MIT App Inventor is used. It is assumed that the reader is familiar with the article titled 'Make a Digital Clock on Your Android Mobile Device,' published in the January 2024 issue of Open Source For You magazine. Power up the HC-05 and Arduino combination. Open the Android phone and navigate to settings > Bluetooth > available devices.

If 'rarely used devices (1)' appears below this, click on it. The HC-05 may be listed. Click on it and pair it. The pairing code is either 0000 or 1234. Close Settings.

MIT app inventor. Log in to the MIT App Inventor as mentioned in the earlier article. It is recommended to use the same words to ensure easy comparison between the screenshots provided and the user's screen.

Click on Projects > Start new project. Give the device a name. For instance, 'Bluetooth_Control' (avoid using 'Bluetooth' as there